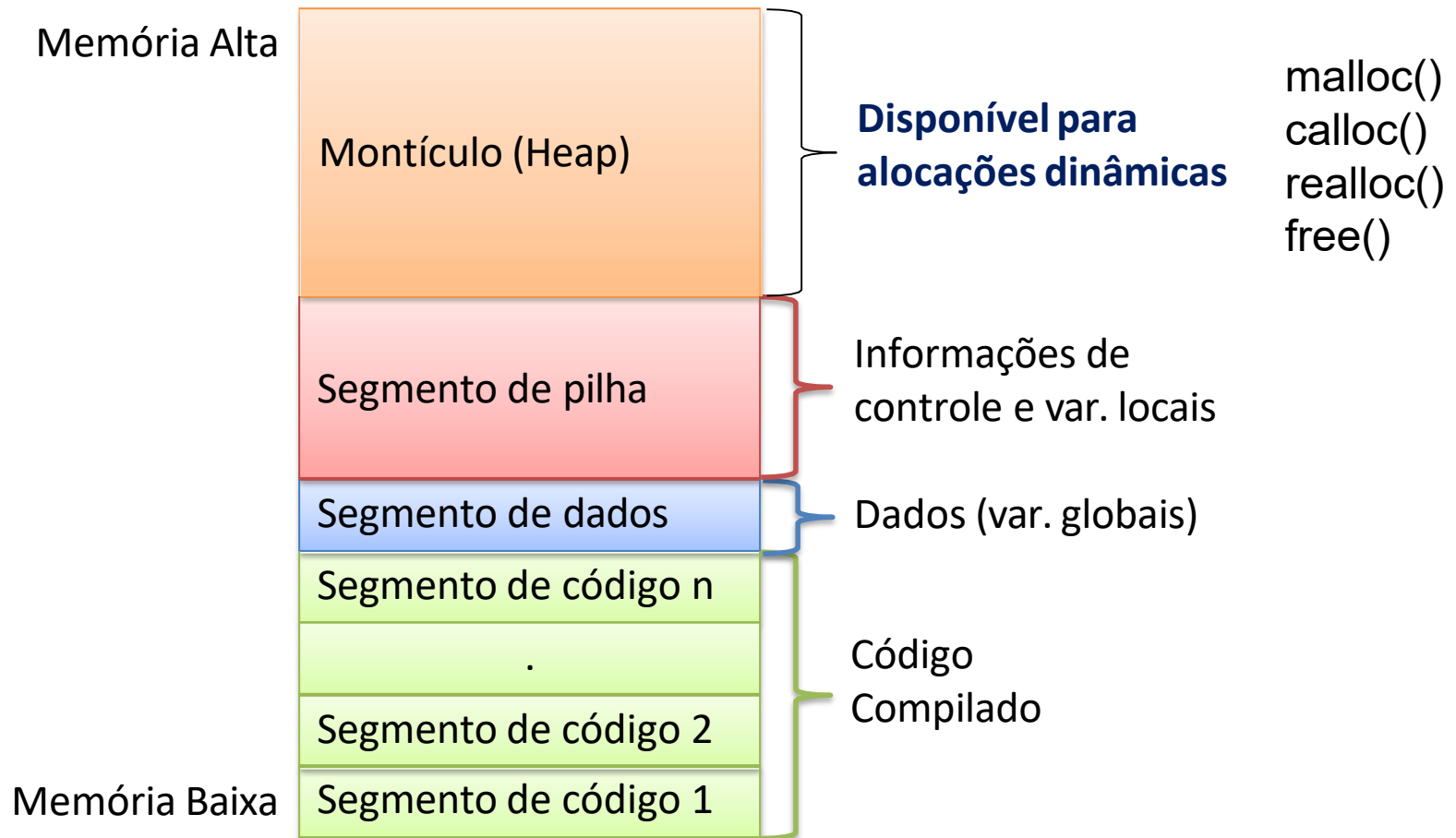


Estrutura de Dados I

Alocação Dinâmica de Memória

- Necessário sempre que a quantidade de memória requerida só é sabida em tempo de execução:
 - Carregamento dos dados de um arquivo;
 - Operações com vetores/matrizes cujas dimensões dependem do contexto do problema;
 - ...

- Mapa de memória de um programa:



- Vantagens:

- Controle do uso de memória: uso sob demanda;

```
int *idades;
```

```
idades = malloc(quantidade * sizeof(int));
```

- Cuidados:

- Gerenciar lixo;

quando usamos `malloc`, `calloc` ou `realloc`, a memória fica reservada no **heap** até que seja liberada com `free`

- Implementação em C/C++:

- Via ponteiros;

```
int *p;
```

```
p = malloc(sizeof(int));
```

- Funções/Operadores de alocação e desalocação;

```
malloc()  
calloc()  
realloc()  
free()
```

- malloc:

```
void* malloc (size_t qtde);
```

qtde: quantidade de bytes a serem alocados.

tipo do dado *nome = (tipo do dado *) malloc(sizeof(tipo do dado) * tamanho);

A função `malloc` significa **memory allocation**, ou seja, alocação de memória.

Ex.: `int *vetor = (int *) malloc(sizeof(int) * 10);`

`int *vetor` cria um ponteiro para inteiro.

`malloc(...)` reserva espaço na memória.

`sizeof(int)` calcula quantos bytes um inteiro ocupa.

`sizeof(int) * 10` reserva espaço suficiente para 10 inteiros.

O `(int *)` converte o retorno de `malloc` para ponteiro de inteiro.

Em outras palavras, o programa está dizendo:

“Reserve memória suficiente para armazenar 10 valores inteiros e faça o ponteiro `vetor` apontar para esse espaço.”

- calloc:

```
void* calloc (size_t qtde, size_t tam);
```

qtde: quantidade de elementos a serem alocados.

tam: tamanho (em bytes) de cada elemento.

```
tipo do dado * nome = (tipo do dado *) calloc (tamanho, sizeof(tipo do dado));
```

A função calloc significa **contiguous allocation**, ou alocação contígua.

```
int *vetor = (int *) calloc(10, sizeof(int));
```

“Reserve memória para 10 elementos do tipo inteiro.”

malloc: Aloca memória, mas não inicializa os valores.

calloc: Aloca memória e inicializa tudo com zero.

- free:

```
void free(void* ptr);
```

ptr: ponteiro para a área de memória a ser desalocada.

free: liberar uma área de memória que foi alocada dinamicamente.

- *#include <stdlib.h>*
 - malloc, calloc
 - free

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *vetor;

    vetor = (int *) calloc(5, sizeof(int));

    if (vetor == NULL) {
        printf("Erro ao alocar memoria.\n");
        return 1;
    }

    vetor[0] = 10;
    vetor[1] = 20;

    printf("%d\n", vetor[0]);
    printf("%d\n", vetor[1]);

    free(vetor);           //Isso evita vazamentos de memória (memory leak).
    vetor = NULL;         //Boa prática para evitar ponteiros pendurados (dangling pointers).

    return 0;
}
```



```
// Exemplo - C: malloc, calloc, free
#include <stdlib.h>    // Biblioteca de: malloc, calloc, free

int main ()
{
    int *Vet1, *Vet2;
    Vet1 = (int*) malloc (100*sizeof(int));
    Vet2 = (int*) calloc (100, sizeof(int));
    free (Vet1);
    free (Vet2);
    return 0;
}
```

//Declaração dos ponteiros
//Alocação de memória
//Alocação de memória
//Desalocação de memória
//Desalocação de memória

malloc
Vet1[0]
Vet1[1]
...
Vet1[99]

calloc
Vet2[0] = 0
Vet2[1] = 0
...
Vet2[99] = 0

Um programa mais seguro seria ideal verificar se a alocação deu certo:

```
if (Vet1 == NULL) {  
    printf("Erro ao alocar memoria para Vet1.\n");  
}  
if (Vet2 == NULL) {  
    printf("Erro ao alocar memoria para Vet2.\n");  
}
```

- new:

`tipoD *nome = new tipoD[qtde];`

*qtde: quantidade de elementos do tipo **tipoD** a serem alocados.*

`tipoD *nome = new tipoD[tamanho];`

- delete[]:

`void delete[];`

`delete[] Vetor;`

Vetor: ponteiro alocado pelo comando new

- **new e delete são implementados como operadores** da própria linguagem C++. Isto significa que não há necessidade de incluir nenhuma biblioteca ao código.

```
// Exemplo – C++: new e delete[]  
int main ()  
{  
    int pind, ptam;  
    float *pVetor;                //Declaração dos ponteiros  
    printf ("Digite a qtde de elementos: ");  
    scanf ("%d",ptam);  
    pVetor = new float[ptam];      //Alocação de memória  
    for(pind=0;pind<ptam;pind++)  
        pVetor[pind] = ((float)pind)/ptam;  
    for(pind=0;pind<ptam;pind++)  
        printf ("%f",pVetor[pind]);  
    delete[] pVetor;              //Desalocação de memória  
    return 0;  
}
```

- Um vetor, seja alocado estaticamente, seja alocado dinamicamente, não guarda informações sobre o seu tamanho;
- Sempre que criar funções que recebem vetores como parâmetros de entrada, lembre-se de passar o tamanho do vetor também.

```
int tamanho = sizeof(notas) / sizeof(notas[0]);
```

```
int *vetor = malloc(5 * sizeof(int));
```

```
#include <stdio.h>
```

```
void imprimirVetor(int v[], int tamanho) {  
    for (int i = 0; i < tamanho; i++) {  
        printf("%d ", v[i]);  
    }  
}
```

```
int main() {  
    int numeros[5] = {10, 20, 30, 40, 50};
```

```
    imprimirVetor(numeros, 5);
```

```
    return 0;  
}
```

```
#include <stdio.h>
#include <stdlib.h>

void preencherVetor(int *v, int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        v[i] = i + 1;
    }
}

int main() {
    int tamanho = 5;

    int *vetor = malloc(tamanho * sizeof(int));

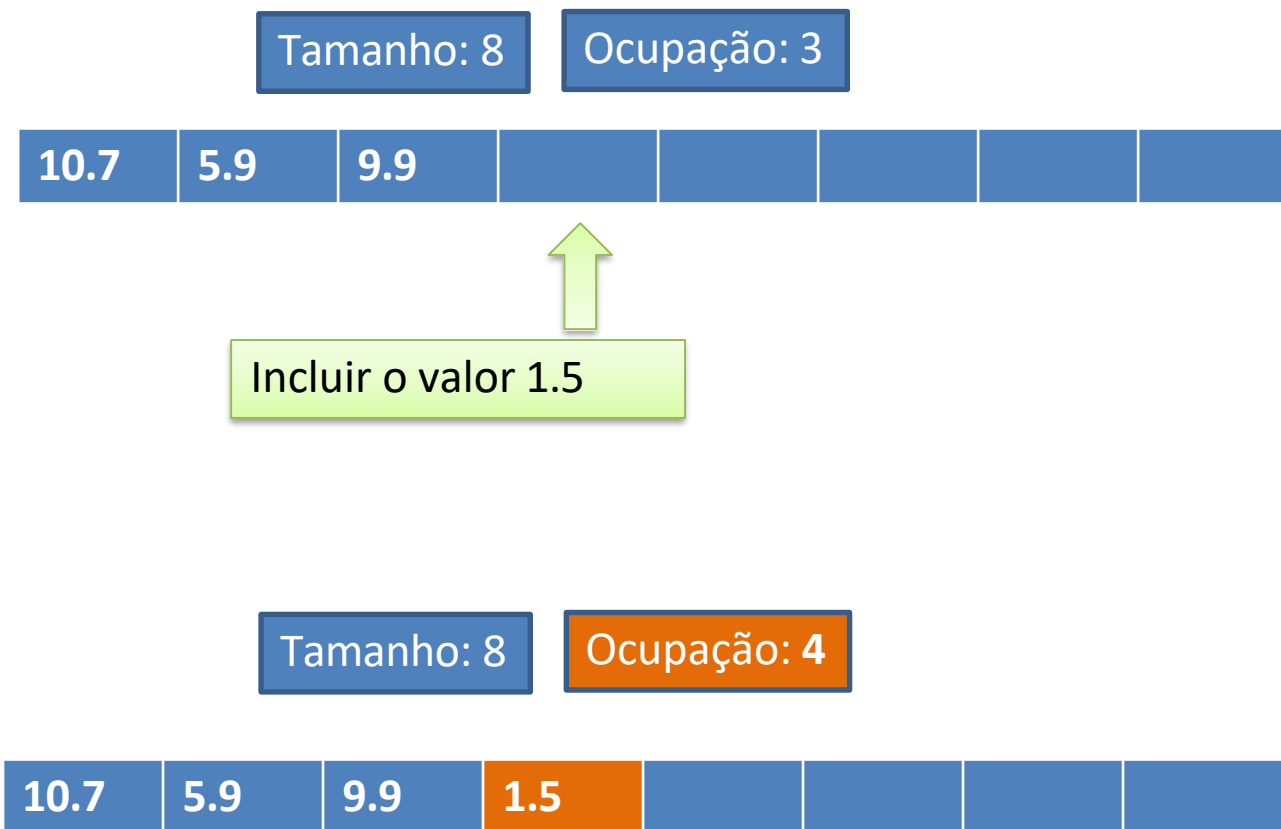
    if (vetor == NULL) {
        printf("Erro ao alocar memoria.\n");
        return 1;
    }

    preencherVetor(vetor, tamanho);

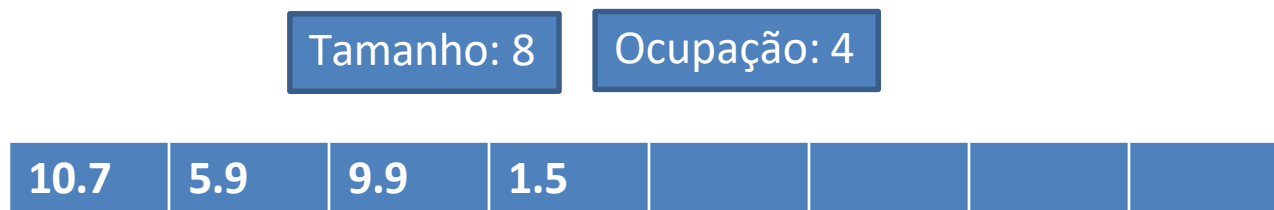
    free(vetor);

    return 0;
}
```

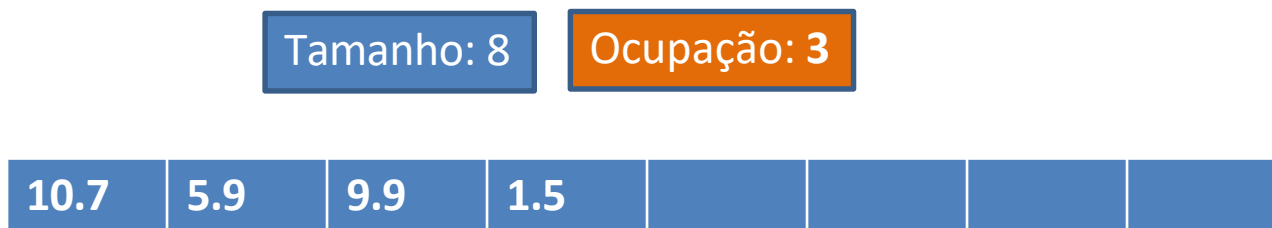
- Inclusão:



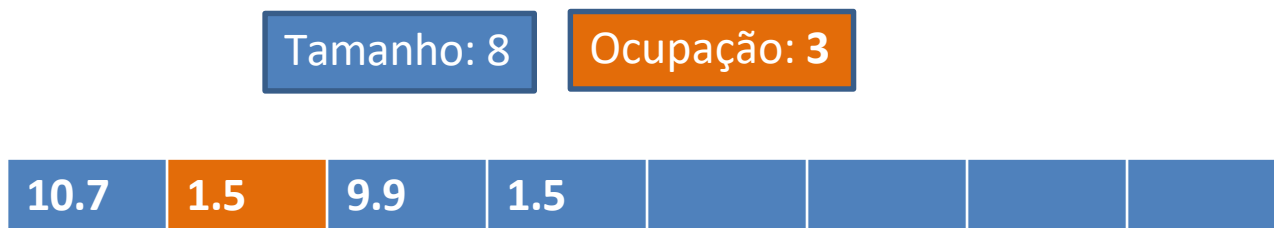
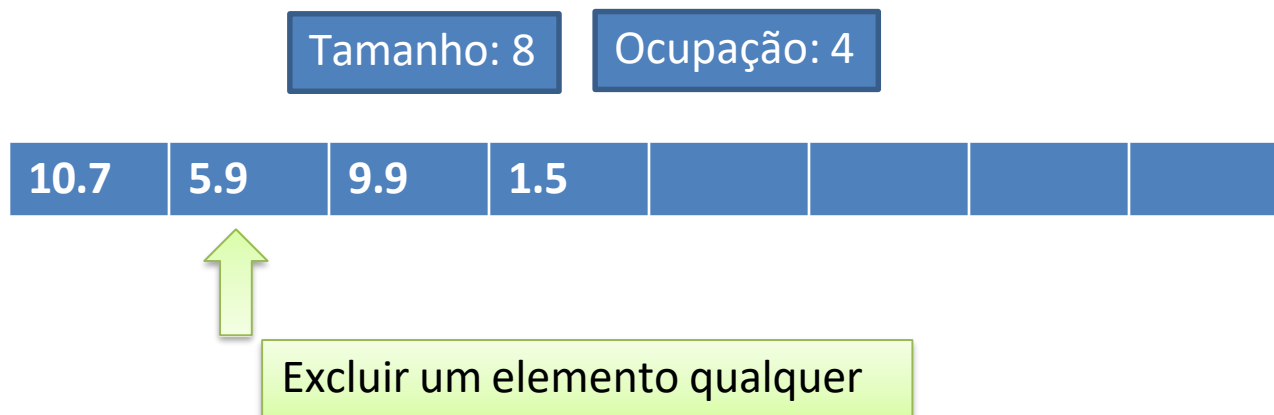
- Exclusão:



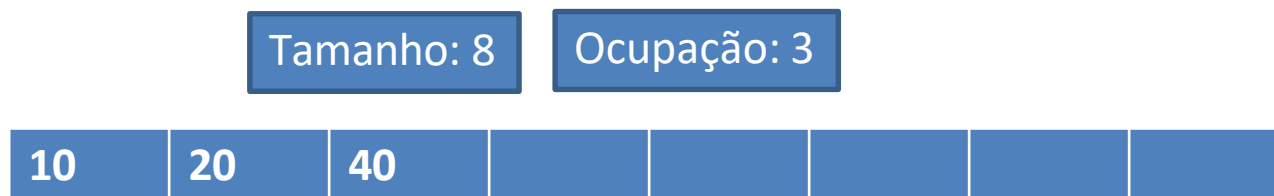
↑
Excluir o último elemento



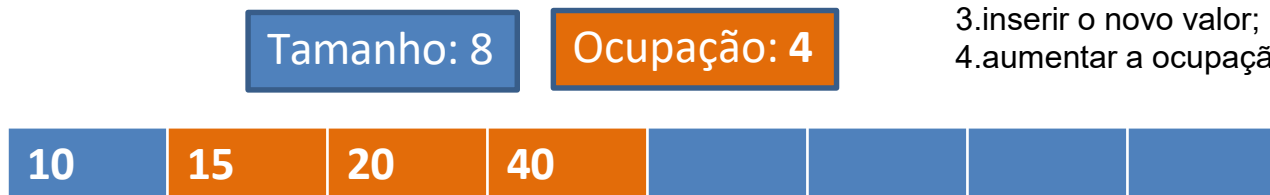
- Exclusão:



- Inclusão:



Incluir o valor 15 na posição 1.

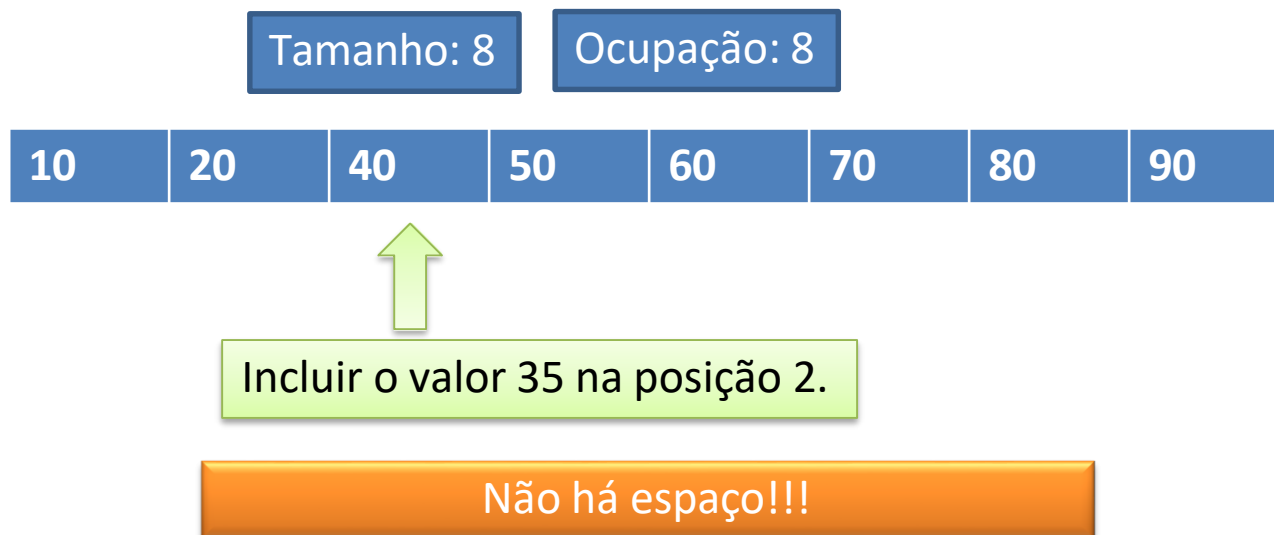


Em um **vetor ordenado**, a inclusão exige mais cuidado.

Não basta colocar o novo valor na próxima posição livre. É preciso:

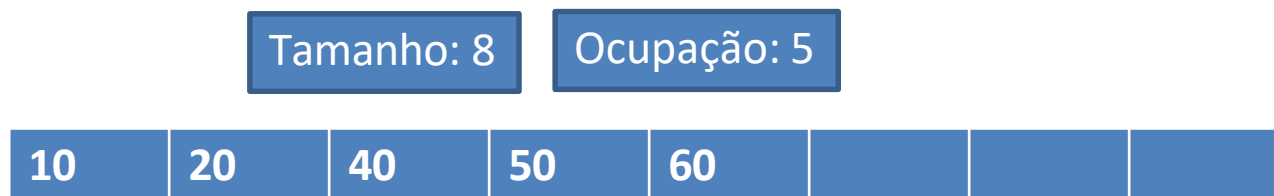
- 1.encontrar a posição correta;
- 2.deslocar os elementos maiores para a direita;
- 3.inserir o novo valor;
- 4.aumentar a ocupação.

- Inclusão:

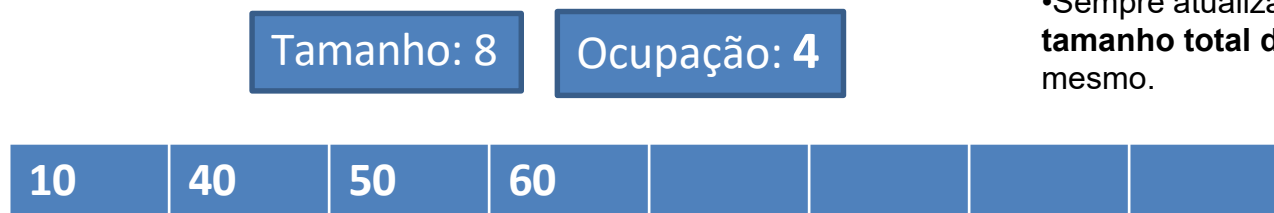


```
if (ocupacao < tamanho) {
    // pode inserir
} else {
    printf("Nao ha espaco!\n");
}
```

- Exclusão:



Excluir o elemento da posição 1.



- **Vetor ordenado:** ao excluir, precisamos **deslocar elementos à direita** para manter a ordem.
- **Vetor desordenado:** poderia apenas substituir o elemento removido pelo último elemento válido.
- Sempre atualizar a **ocupação**, mas o **tamanho total do vetor** permanece o mesmo.